CIS 4930
ML for Computer Systems
Project Proposal

Team Members:

Riyon Praveen, Vishakha Tiwari, Sevara Badriddinova, Tuan Khang Phan

Problem:

Database systems rely on index structures to locate data quickly. The two most widely used are B+trees, which support both exact-match lookups and range queries, and hash tables, which provide fast exact-match lookups but cannot answer range queries. Both are general-purpose: they behave the same regardless of how the data is distributed, and they do not take advantage of patterns in the data.

Kraska et al. [1] proposed *learned indexes*, which replace the tree with a model that learns the relationship between each key and its position in a sorted array. To find a key, a learned index predicts its approximate position and then searches a small range around that prediction. The authors reported learned indexes running up to 70% faster than cache-optimized B+trees while using an order of magnitude less memory.

These results were obtained on hardware that is now several years old. Modern processors have substantially larger caches and more varied architectures (for example, x86 and ARM), and index performance depends heavily on how often a lookup must wait on main memory. It is therefore unclear whether the reported advantages still hold. In addition, learned indexes have known weaknesses, including irregular data distributions, insert-heavy workloads, and build time, that headline results do not always capture.

Project Goal:

Our goal is to determine when and why learned indexes outperform traditional indexes on modern hardware.

We will conduct a controlled benchmark comparing several learned indexes against traditional baselines, using both real-world and synthetic datasets. The learned indexes are the Recursive Model Index (RMI) [1], the Piecewise Geometric Model index (PGM-index) [3], RadixSpline [4], and ALEX, an updatable learned index [5]. The traditional baselines are a B+tree, a hash table, and binary search on a sorted array. In addition to measuring performance, we will use hardware performance counters to explain our results in terms of processor behavior, such as cache misses and branch mispredictions. We will build on the Search On Sorted Data (SOSD) benchmark [2], an open-source framework for evaluating learned indexes. We will extend it with additional baselines, workloads, and hardware-level analysis.

Objectives:

1. **Compare lookup performance.** Measure the average and 99th-percentile (p99) lookup latency, as well as throughput, of each index on at least four datasets with different key distributions.
2. **Evaluate the trade-off between memory and speed.** Measure the memory footprint of each index and how its lookup latency changes as the index is tuned larger or smaller. For learned indexes this means the number of model segments; for B+trees it means the node size.
3. **Explain performance using hardware counters.** Use the Linux `perf` tool to measure cache misses, branch mispredictions, and instructions per lookup, and use these measurements to explain the performance differences observed in Objectives 1 and 2.
4. **Identify where learned indexes fall short.** Evaluate each index on workloads where learned indexes are expected to struggle: datasets that are difficult to model, insert-heavy workloads, range queries, and index build time.
5. **Assess the effect of modern hardware.** Repeat key experiments on at least two machines with different processor architectures, and with dataset sizes both smaller and larger than the processor cache. The goal is to determine whether the conclusions of [1] still hold on current hardware.

References:

[1] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. "The Case for Learned Index Structures." *SIGMOD*, 2018.

[2] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska. "Benchmarking Learned Indexes." *PVLDB* 14(1), 2020.

[3] P. Ferragina and G. Vinciguerra. "The PGM-index: A Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds." *PVLDB* 13(8), 2020.

[4] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann. "RadixSpline: A Single-Pass Learned Index." *aiDM Workshop at SIGMOD*, 2020.

[5] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann, D. Lomet, and T. Kraska. "ALEX: An Updatable Adaptive Learned Index." *SIGMOD*, 2020.

AI was used to assist with brainstorming and refining the project proposal.